

US Accidents Severity Prediction

A step-by-step plan from raw data to final report

Item	Summary
Objective	Predict the severity (1-4) of a US traffic accident using only information available when the accident is first reported: time, location, weather and nearby road features.
Dataset	US Accidents (2016-2023), Moosavi et al., Kaggle. About 7.7 million records, 46 columns, 49 US states.
ML task	Supervised multi-class classification (4 classes, heavily imbalanced).
Main metric	Macro F1-score , supported by per-class recall and balanced accuracy.
Working data	Stratified sample of about 500,000 rows (the full file is about 3 GB).
Models	Dummy baseline → Logistic Regression → Decision Tree → Random Forest → LightGBM / XGBoost.
Tools	Python, pandas, scikit-learn, imbalanced-learn, LightGBM, XGBoost, SHAP, matplotlib, seaborn, folium.
Duration	8 weeks, 12 phases (Section 4). Compress to fit your deadline.

The one rule of this project. A feature is allowed only if its value is known **at the moment the accident is reported**. Columns measured afterwards (end time, affected distance, the text description) reveal the answer and must be removed (Phase 3).

Contents

#	Section	What it covers
1	Project definition	Problem, target, objectives, success criteria
2	Dataset overview	Facts, column groups, citation
3	Step-by-step plan	Phases 1-12, each with a goal, tasks and output
4	Timeline	8-week schedule
5	Team roles	Suggested split for a 3-4 person team
6	Risks and mitigation	What can go wrong and what to do
7	Deliverables checklist	What you hand in
8	Final report structure	Chapter outline
9	References	Papers and data source

Phase overview

Phase	Name	Main output
1	Setup and organisation	Environment, GitHub repo, folder structure
2	Get the data and sample it	<code>sample.parquet</code> (~500K rows)
3	Leakage audit and column selection	List of kept and dropped columns with reasons
4	Data cleaning	<code>clean.parquet</code> and a cleaning log
5	Exploratory data analysis (EDA)	10-15 charts with written insights
6	Feature engineering	Model-ready feature table
7	Train / validation / test split	Three fixed sets, plus a time-based check
8	Handling class imbalance	Chosen strategy, justified on validation data
9	Modelling and tuning	4-5 trained models, tuned best model
10	Evaluation	Model comparison table, confusion matrix
11	Interpretation	Feature importance, SHAP, insights, limitations
12	Report and presentation	Final PDF report, slides, clean repo

1. Project definition

1.1 Problem statement

Traffic management centres and navigation apps need to know quickly which accidents will cause long delays, so they can send response teams and reroute traffic. The project answers one question:

Given when, where and under what conditions an accident happens, can we predict how severe its impact on traffic will be?

1.2 The target variable: Severity

Severity	Meaning (dataset definition)	Approx. share
1	Least impact on traffic (short delay)	Small minority
2	Moderate impact	About 80%
3	Significant impact	Second most common
4	Highest impact on traffic (long delay)	About 2.6%

Important wording for the report: Severity measures the **impact on traffic** (delay), not injuries or deaths. Do not describe it as "how dangerous" the accident was. Confirm the exact class shares on your own sample in Phase 2.

1.3 Objectives

- Build a clean, reproducible working sample of the dataset.
- Explore and visualise when, where and under what conditions severe accidents happen.
- Train and compare at least four classification models against a baseline.
- Handle the class imbalance and justify the chosen method.
- Explain which factors drive severity, and state the limitations honestly.

1.4 Success criteria

Predicting "Severity 2" for every accident already gives about 80% accuracy, so accuracy alone is meaningless here. The same trivial model has a macro F1 of about:

$$(0.89 + 0 + 0 + 0) / 4 \approx 0.22$$

because it scores $F1 \approx 0.89$ on class 2 and zero on the other three. The project succeeds if:

- The best model clearly beats the dummy baseline on **macro F1** (well above 0.22).
- Recall on **Severity 4** is reported and is meaningfully above zero.
- No leakage columns are used (Phase 3 is documented).
- Results are reproducible from the repository with a fixed random seed.

2. Dataset overview

Property	Value
Name	US Accidents (2016 – 2023): A Countrywide Traffic Accident Dataset
Author	Sobhan Moosavi et al. (Ohio State University)
Source	Kaggle: kaggle.com/datasets/sobhanmoosavi/us-accidents
Records	About 7.7 million accidents
Period	February 2016 – March 2023
Coverage	49 US states (contiguous United States)
Columns	46
File	One CSV of about 3 GB
License	CC BY-NC-SA 4.0: academic use allowed, citation required

2.1 Column groups

Group	Columns
Target	Severity
IDs and source	ID, Source
Time	Start_Time, End_Time [△] , Weather_Timestamp, Timezone
Location (GPS)	Start_Lat, Start_Lng, End_Lat [△] , End_Lng [△] , Distance(mi) [△]
Address	Street, City, County, State, Zipcode, Country, Airport_Code
Text	Description [△]
Weather	Temperature(F), Wind_Chill(F), Humidity(%), Pressure(in), Visibility(mi), Wind_Direction, Wind_Speed(mph), Precipitation(in), Weather_Condition
Road features (True/False)	Amenity, Bump, Crossing, Give_Way, Junction, No_Exit, Railway, Roundabout, Station, Stop, Traffic_Calming, Traffic_Signal, Turning_Loop
Light conditions	Sunrise_Sunset, Civil_Twilight, Nautical_Twilight, Astronomical_Twilight

[△] = leakage risk, removed in Phase 3.

2.2 Required citation

The dataset authors ask users to cite both papers listed in Section 9. Put them in your report's reference list from the start.

3. Step-by-step plan

Each phase has a goal, a list of tasks and a concrete output. Do not start a phase until the output of the previous one exists and is saved in the repository.

PHASE 1 Setup and team organisation

Goal Everyone can run the same code on the same data.

Output Working environment, shared GitHub repository, agreed folder structure.

- Install Python 3.10+ with Jupyter or VS Code. Google Colab (free) also works for a 500K-row sample.
- Libraries: pandas numpy pyarrow matplotlib seaborn folium scikit-learn imbalanced-learn lightgbm xgboost shap.
- Create a GitHub repository and save the library list in `requirements.txt`.
- Fix one random seed (e.g. 42) and use it everywhere.
- Agree on the folder structure below and on notebook naming.

```
us-accidents-project/
├── data/raw/           # original CSV (NOT pushed to GitHub, too big)
├── data/processed/     # sample.parquet, clean.parquet, features.parquet
├── notebooks/
│   ├── 01_load_and_sample.ipynb
│   ├── 02_cleaning.ipynb
│   ├── 03_eda.ipynb
│   ├── 04_features.ipynb
│   ├── 05_modelling.ipynb
│   └── 06_evaluation.ipynb
├── reports/figures/
├── requirements.txt
└── README.md
```

PHASE 2 Get the data and build a working sample

Goal A manageable, representative dataset that loads in seconds.

Output `data/processed/sample.parquet` with about 500,000 rows and the same severity proportions as the full data.

- Create a Kaggle account and download the dataset (CSV of about 3 GB).
- Do not load the whole file into pandas on a laptop: it needs many GB of RAM. Read it in chunks and keep a stratified 7% sample (about 540K rows).
- Save the sample as Parquet: it is much smaller and loads far faster than CSV.
- Check that the sample matches the full data: Severity shares, rows per year and per state.
- Record the full-data class counts; you will quote them in the report.

```
import pandas as pd

parts = []
for chunk in pd.read_csv("data/raw/US_Accidents_March23.csv", chunksize=500_000):
    # keep 7% of every Severity class in every chunk -> stratified sample
    parts.append(chunk.groupby("Severity", group_keys=False)
                  .sample(frac=0.07, random_state=42))

df = pd.concat(parts, ignore_index=True)
print(df.shape)
print(df["Severity"].value_counts(normalize=True))
df.to_parquet("data/processed/sample.parquet", index=False)
```

Use the exact CSV file name from your Kaggle download if it differs.

PHASE 3 Leakage audit and column selection

Goal Remove every column that would not be known when the accident is reported.

Output A table of kept and dropped columns with a reason for each. It goes into the report.

Ask of every column: **could this value be known at the moment the accident is first reported?** If not, drop it.

Column	Decision	Reason
End_Time	Drop	Known only after the road is cleared. Duration $\approx$ delay $\approx$ severity.
End_Lat, End_Lng	Drop	Describe the affected stretch of road, which is measured afterwards.
Distance(mi)	Drop	Length of road affected. This is part of how severity is judged.
Description	Drop	Free text often says "road closed" or "lanes blocked", which reveals severity.
ID	Drop	Identifier only, no information.
Country	Drop	Always "US" (constant).
Turning_Loop	Drop if constant	Check with <code>nunique()</code> ; drop if it has one value.
Weather_Timestamp, Timezone, Airport_Code	Drop	Redundant with Start_Time and location.
Source	Keep for analysis	Different data providers may record severity differently. Compare the model with and without it (Phase 9).
City, County, Zipcode, Street	Keep with care	Very many unique values. Encode them in Phase 6, don't one-hot encode them.
All weather, road, light and time columns	Keep	Known at report time.

PHASE 4 Data cleaning

Goal	A consistent, correctly typed dataset with no impossible values.
Output	<code>clean.parquet</code> and a cleaning log (rows and columns before and after each step).

4.1 Types and duplicates

- Parse times: `pd.to_datetime(df["Start_Time"], format="mixed")`. Some rows have fractional seconds, which break a single fixed format.
- Convert the True/False road-feature columns to 0/1.
- Drop exact duplicate rows and record how many were removed.

4.2 Missing values

Start by measuring them with `df.isna().mean().sort_values()`, then apply a rule per column type:

Situation	Action
Column mostly empty (e.g. > 40%)	Drop the column.
Wind_Chill(F)	Drop: many missing values, and it is nearly a copy of Temperature.
Precipitation(in)	Fill with 0 and add a <code>precip_missing</code> flag. State this assumption in the report.
Other numeric weather (Temperature, Humidity, Pressure, Visibility, Wind_Speed)	Median imputation, fitted on the training set only (inside the Pipeline, Phase 9).
Categorical (Weather_Condition, Wind_Direction, light columns)	Fill with "Unknown".
Few rows missing City, Zipcode or light columns	Drop those rows.

4.3 Outliers and impossible values

- Inspect ranges with `df.describe()` and box plots.
- Set physically impossible values to missing, then impute them. Examples: temperatures above 130 °F or below −50 °F, wind speeds above 200 mph, visibility above 100 miles.
- Document every threshold you choose and why.

4.4 Standardise categories

- **Wind_Direction:** merge spellings such as "Calm"/"CALM", "West"/"W", "South"/"S", "Variable"/"VAR" into 16 compass points plus CALM and VAR.
- **Weather_Condition** has over 100 labels. Group them into about 8: Clear, Cloudy, Rain, Heavy rain / Thunderstorm, Snow / Ice, Fog / Haze / Smoke, Windy, Other.

PHASE 5 Exploratory data analysis (EDA)

Goal Understand the data and find the story before modelling.

Output 10-15 clear, labelled charts in reports/figures/, each with a 1-2 sentence insight.

#	Question	Suggested chart
1	How imbalanced is Severity?	Bar chart of class counts and percentages
2	When do accidents happen?	Counts by hour, weekday, month and year
3	Are rush-hour or night accidents more severe?	100% stacked bar of Severity by hour
4	Where do accidents happen?	Top 10 states and cities; folium heat map on a 50K subsample
5	Does weather matter?	Severity share per weather group; box plots of visibility and temperature by Severity
6	Do road features matter?	% of Severity 3-4 near a junction, signal, crossing or stop
7	Day or night?	Severity share by Sunrise_Sunset
8	Are numeric features related?	Correlation heat map
9	Is the data stable over time?	Severity share by year and by Source (checks drift and reporting differences)

Every chart needs a title, axis labels and units. Write down what it shows. A chart without an insight does not belong in the report.

PHASE 6 Feature engineering

Goal Turn raw columns into features that carry signal.

Output A model-ready feature table and a data dictionary for it.

New feature	How to build it	Why it may help
hour, weekday, month, year	From Start_Time	Traffic patterns depend on time
is_weekend	weekday $\geq$ 5	Weekend traffic differs
is_rush_hour	Weekday and hour in 7-9 or 16-19	Congestion makes delays longer
is_night	Sunrise_Sunset == "Night"	Lower visibility, slower response
season	Month $\rightarrow$ Winter / Spring / Summer / Autumn	Snow and ice in winter
weather_group, bad_weather	Grouped Weather_Condition; flag for rain, snow, fog or storm	Fewer, stronger categories
road_feature_count	Sum of the road-feature flags	Complexity of the location
is_highway	Street contains "I-", "Interstate", "Hwy", "Highway", "US-", "Fwy"	Highway accidents often cause bigger delays
State	One-hot encoding (49 values)	Regional differences
City, County	Frequency encoding (accident count per city)	Handles thousands of values

New feature	How to build it	Why it may help
location_cluster	KMeans (e.g. 50 clusters) on Start_Lat / Start_Lng	Captures regions and hotspots

Put all encoders, imputers and scalers inside a scikit-learn Pipeline with a ColumnTransformer. They are then fitted on training data only. Only Logistic Regression needs scaling; tree models do not.

PHASE 7 Train / validation / test split

Goal	Evaluate the model on data it has never seen.
Output	Three fixed sets saved to disk, plus a time-based check set.

- **Main split:** stratified random 70% train / 15% validation / 15% test, stratified on Severity.
- **Split before** any imputation, encoding, resampling or scaling is fitted.
- Use the validation set for all choices: imbalance method, model and hyperparameters.
- Use the **test set once**, at the very end, for the final numbers.
- **Robustness check (time-based):** train on 2016–2021 and test on 2022–2023. This shows whether the model works on future data. Expect a lower score and discuss why (drift, changes in reporting).

PHASE 8 Handling class imbalance

Goal	Make the model learn the rare classes (1 and 4) instead of always predicting 2.
Output	A chosen strategy, justified by validation macro F1.

Strategy	How	Notes
Class weights (start here)	class_weight="balanced"	Easy, no data duplication, supported by most models
Random undersampling	Reduce Severity 2 rows in the training set only	Fast, smaller training set, loses some data
SMOTE oversampling	imbalanced-learn, training set only , after the split	Slow on large data; never apply it to validation or test
Decision-threshold tuning	Adjust class probability cut-offs on validation	Improves recall on rare classes

Compare at least two strategies. **Optional extra experiment:** a binary target, "high impact" (Severity 3–4) against "low impact" (1–2). It is a good fallback if time is short.

PHASE 9 Modelling and tuning

Goal Train models of increasing complexity and tune the best one.

Output Trained models, a tuned best model, and training times.

Step	Model	Purpose
0	DummyClassifier (most frequent)	Reference point: macro F1 $\approx$ 0.22
1	Logistic Regression (multinomial, balanced)	Linear baseline, interpretable coefficients
2	Decision Tree	Simple non-linear model; can be drawn in the report
3	Random Forest	Strong, robust ensemble
4	LightGBM or XGBoost	Usually the best on tabular data, and fast on 500K rows
5 (optional)	MLP neural network	Comparison with a neural approach

- Tune with RandomizedSearchCV (3-fold StratifiedKFold, scoring="f1_macro") on a 100–200K training subsample to save time.
- LightGBM parameters worth tuning: num_leaves, learning_rate, n_estimators, min_child_samples, subsample, colsample_bytree.
- Ablation: train the best model with and without **Source**, and report the difference.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from lightgbm import LGBMClassifier

pre = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore", min_frequency=0.01), cat_cols),
])
model = Pipeline([
    ("pre", pre),
    ("clf", LGBMClassifier(class_weight="balanced", n_estimators=500,
                          learning_rate=0.05, random_state=42, n_jobs=-1)),
])
model.fit(X_train, y_train)
```

PHASE 10 Evaluation

Goal Measure performance honestly, especially on the rare classes.

Output A model comparison table, a normalised confusion matrix and an error analysis.

10.1 Metrics

- **Macro F1** (main metric): the average of the per-class F1 scores, so every class counts equally.
- Per-class precision, recall and F1 (classification_report).
- Balanced accuracy, weighted F1, and one-vs-rest ROC-AUC.
- Report accuracy too, but explain why it is misleading here (predicting 2 every time gives about 80%).

10.2 Results table template

Model	Macro F1	Balanced acc.	Recall Sev 4	ROC-AUC (OvR)	Train time
Dummy	≈ 0.22	0.25	0.00	0.50	—
Logistic Regression					
Decision Tree					
Random Forest					
LightGBM / XGBoost					

10.3 Error analysis

- Normalised confusion matrix: which classes get mixed up (typically 1 with 2, and 3 with 4)?
- Where and when does the model fail: by state, hour, weather and Source?
- Compare the random-split score with the time-based score from Phase 7.

PHASE 11 Interpretation

Goal Explain what drives severity and what the model can and cannot say.

Output Importance plots, 3–5 key insights, and a limitations paragraph.

- Built-in feature importance (gain) and **permutation importance** on the test set.
- **SHAP** summary plot for the best model (computed on 5–10K rows for speed).
- Turn findings into plain statements, e.g. "accidents on highways during rush hour are much more likely to be Severity 4". Support each with a number.
- **Limitations:** severity is traffic impact, not injury; providers may record severity differently; the model uses a sample; the data ends in March 2023; coverage varies by state.

PHASE 12 Report and presentation

Goal Communicate the work clearly and make it reproducible.

Output Final PDF report, 10–12 slides, clean GitHub repository.

- Write the report following the structure in Section 8.
- Slides: problem → data → leakage → EDA highlights → models → results → insights → limitations.
- Repository: README with how to run, requirements.txt, numbered notebooks, fixed seeds.
- **Optional bonus:** a small Streamlit app where a user enters time, place and weather and gets a predicted severity.
- Rehearse the presentation and prepare for questions on leakage, imbalance and metric choice.

4. Timeline (8 weeks)

Week	Phases	Work	Checkpoint
1	1-2	Setup, download, stratified sample	sample.parquet exists; class shares checked
2	3-4	Leakage audit, cleaning	clean.parquet and cleaning log
3	5	EDA	10-15 charts with insights
4	6-7	Feature engineering, data split	Feature table, fixed splits. Freeze the scope.
5	8-9	Baseline models, imbalance strategies	Dummy, Logistic Regression and Decision Tree results
6	9	Random Forest, LightGBM / XGBoost, tuning	Tuned best model
7	10-11	Test-set evaluation, time-based check, SHAP	Final results table and insights
8	12	Report, slides, repo clean-up, rehearsal	Submission

If your deadline is shorter, merge weeks 2-3 and 5-6, and skip the optional items.

5. Team roles (3-4 people)

Role	Main responsibility	Phases
Data engineer	Loading, sampling, cleaning, data dictionary	2, 3, 4
Data analyst	EDA, charts, maps, insights	5, 11
ML engineer	Features, pipelines, models, tuning, evaluation	6-10
Report lead	Report writing, slides, README, reproducibility checks	1, 12

Everyone reviews each other's notebooks. With 3 people, merge the analyst and report lead roles.

6. Risks and mitigation

Risk	Mitigation
The 3 GB file does not fit in memory	Chunked stratified sampling, Parquet, Google Colab, or Polars
Data leakage inflates the scores	Phase 3 audit and the "known at report time" rule; pipelines fitted on training data only
Imbalance hides poor minority-class performance	Macro F1, per-class recall, class weights
Providers record severity differently (Source bias)	EDA by Source; ablation with and without Source
Distribution changes over the years	Time-based test (Phase 7); discuss it in limitations
Training or tuning is too slow	Tune on a subsample, use LightGBM, n_jobs=-1
Schedule slips	Freeze the scope in week 4; binary-target fallback (Phase 8)

7. Deliverables checklist

- ☐ Stratified sample and cleaned dataset (Parquet) with a data dictionary
- ☐ Leakage audit table (kept and dropped columns with reasons)
- ☐ Cleaning log (rows and columns before and after each step)
- ☐ Numbered notebooks 01-06 that run from top to bottom
- ☐ EDA figures with written insights
- ☐ Model comparison table (validation and test)
- ☐ Confusion matrix, per-class report, time-based robustness result
- ☐ Feature importance and SHAP plots with interpretation
- ☐ Final written report (PDF)
- ☐ Presentation slides
- ☐ GitHub repository with README and requirements.txt

8. Final report structure

#	Chapter	Content
—	Abstract	Problem, data, best model, main result, in about 150 words
1	Introduction	Motivation, problem statement, objectives
2	Dataset	Source, size, columns, target definition, citation
3	Data preparation	Sampling, leakage audit, cleaning decisions
4	Exploratory analysis	Key charts and insights
5	Feature engineering	New features and encodings
6	Methodology	Split strategy, imbalance handling, models, tuning, metrics
7	Results	Comparison table, confusion matrix, time-based check
8	Interpretation	Feature importance, SHAP, practical insights
9	Limitations and future work	Honest discussion; ideas such as the full dataset or NLP on descriptions
10	Conclusion	Answer to the problem statement
—	References and appendix	Papers, dataset, link to the code repository

9. References

- [1] Moosavi, S., Samavatian, M. H., Parthasarathy, S., & Ramnath, R. (2019). *A Countrywide Traffic Accident Dataset*. arXiv:1906.05409.
- [2] Moosavi, S., Samavatian, M. H., Parthasarathy, S., Teodorescu, R., & Ramnath, R. (2019). *Accident Risk Prediction based on Heterogeneous Sparse Data: New Dataset and Insights*. Proceedings of the 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems.
- [3] Moosavi, S. *US Accidents (2016 - 2023)* [Dataset]. Kaggle.
<https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents>
- [4] Moosavi, S. *US-Accidents: A Countrywide Traffic Accident Dataset*.
https://smoosavi.org/datasets/us_accidents